Worksheet SOLUTIONS: Uninformed & Heuristic Search

Foundations of Artificial Intelligence – SS 2025 *University of Bremen, Institute for Artificial Intelligence*

Part 1 – Specializations of Search Algorithms

1.1 BFS as a special case of UCS

If all step costs are **equal (e.g. cost = 1)**, UCS always expands the node with the lowest cumulative cost `g(n)`. Since every path of depth `d` has cost `d`, UCS will expand nodes in the same order as BFS — shallowest first. So BFS = UCS with uniform step costs.

1.2 BFS, DFS, UCS as special cases of Best-First Search

Best-First Search expands the node with the lowest `f(n)`. By choosing different `f`:

Algorithm	Evaluation function f(n)
BFS	depth(n) – shallowest first
DFS	–depth(n) – deepest first (or LIFO via stack)
UCS	g(n) – lowest path cost so far

1.3 UCS as a special case of A*

A* uses `f(n) = g(n) + h(n)`. If we set `h(n) = 0` for all nodes, A* reduces to UCS.

Part 2 – Trace UCS and A* by Hand

Graph recap: - S → A: 1, S → B: 3, A → C: 3, B → C: 1, B → G: 2, C → G: 12 - h: S=4, A=3, B=2, C=2, G=0

2.1 UCS (graph-search, sort alphabetically on ties)

Step	Expanded	Frontier after expansion
1	S (g=0)	A/1, B/3
2	A (g=1)	B/3, C/4

Step	Expanded	Frontier after expansion
3	B (g=3)	C/4, G/5
4	C (g=4)	G/5, G/16
5	G (g=5) ✓	—

Path found: S → B → G, **cost = 5**

(C is expanded but the goal is reached via B→G before C→G.)

2.2 A* Search (graph-search)

f(n) = g(n) + h(n)

Step	Expanded	f-values in frontier
1	S (f=0+4=4)	A: 1+3=4, B: 3+2=5
2	A (f=4)	B: 5, C: 4+2=6
3	B (f=5)	C: min(6, 4+2=6)=6, G: 5+0=5
4	G (f=5) ✓	—

Path found: S → B → G, **cost = 5**

A* finds the same optimal path as UCS here but expands **fewer nodes** (skips expanding C before finding the goal).

2.3 Greedy Best-First Search (f = h only)

Step	Expanded	h-values in frontier
1	S (h=4)	A: h=3, B: h=2
2	B (h=2)	A: 3, G: 0, C: 2
3	G (h=0) ✓	—

Path found: S → B → G, **cost = 5**

In this graph GBFS happens to find the optimal path, but this is not guaranteed in general — GBFS is **not** optimal. It is greedy: it always goes where the heuristic says is closest to the goal, ignoring the cost already paid.

2.4 A* with h(C) = 12

The true cost from C to G is 12. Since `h(C) = 12 = true cost`, the heuristic is **still admissible** (never overestimates). A* still finds the optimal path S → B → G with cost 5. The only difference is that if C were ever added to the frontier, its f-value would be even higher, making it even less attractive — which is consistent with correct behavior.

Part 3 – Python BFS and DFS (Solutions)

```python
from collections import deque

def bfs(graph, start, goal):
    queue = deque([[start]])        # each element is a path
    visited = set()
    visited.add(start)
    while queue:
        path = queue.popleft()
        node = path[-1]
        if node == goal:
            return path
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return None

def dfs(graph, start, goal):
    stack = [[start]]
    visited = set()
    visited.add(start)
    while stack:
        path = stack.pop()
        node = path[-1]
        if node == goal:
            return path
        for neighbor in reversed(graph.get(node, [])):
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append(path + [neighbor])
    return None
```

BFS guarantees the **shortest path (fewest edges)**; DFS may find a different path that is longer in terms of edges but happens to reach the goal faster depth-wise.

Part 4 – Python UCS (Solution)

```python
import heapq

def ucs(graph, start, goal):
    # Priority queue entries: (cost, node, path)
    pq = [(0, start, [start])]
    visited = set()
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, cost
        for neighbor, edge_cost in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(pq, (cost + edge_cost, neighbor, path + [neighbor]))
    return None, float('inf')
```

On the Part 2 graph this returns `(['S', 'B', 'G'], 5)` — matching the hand trace.

Part 5 – Python A* (Solution)

```python
import heapq

def astar(graph, heuristic, start, goal):
    # Priority queue entries: (f, g, node, path)
    pq = [(heuristic[start], 0, start, [start])]
    visited = set()
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, g
        for neighbor, edge_cost in graph.get(node, []):
            if neighbor not in visited:
                new_g = g + edge_cost
                new_f = new_g + heuristic[neighbor]
```

```
                heapq.heappush(pq, (new_f, new_g, neighbor, path + [neighbor]))
    return None, float('inf')
```

5.3 Bonus: With `h['C'] = 12`, A* still returns `(['S', 'B', 'G'], 5)`. The heuristic remains admissible, so optimality is preserved.

Part 6 – Reflection

6.1 Admissible vs. Consistent heuristic

A heuristic is **admissible** if it never overestimates the true cost to the goal: `h(n)  h*(n)`. It is **consistent (monotone)** if for every node `n` and successor `n'`: `h(n)  cost(n,n') + h(n')` (triangle inequality). Consistency implies admissibility, but not vice versa.

6.2 A* vs. UCS

A* should be preferred when a good heuristic is available. It uses the heuristic to guide the search toward the goal, typically expanding far fewer nodes than UCS. UCS is appropriate when no meaningful heuristic exists or computing `h(n)` is expensive.

6.3 Graph-search vs. Tree-search and heuristic requirements

In **tree-search**, a node can be re-expanded via a different path, so even if A* first finds a suboptimal path to a node, it may later correct it. Admissibility alone suffices. In **graph-search**, once a node is marked visited it is never re-expanded. If the heuristic is only admissible (not consistent), A* might close a node with a suboptimal g-value before finding the better path. Consistency guarantees that the first time a node is popped from the priority queue, the path is already optimal.

End of solutions. □